SDEV2150

Lesson 11: UI Frameworks with DaisyUI

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

In this lesson, students will:

- Install and configure DaisyUI in a React + Tailwind project
- Use DaisyUI prebuilt components to accelerate UI development
- Customize themes and integrate DaisyUI components into existing layouts

Agenda

- Recap: Tailwind styling from Lesson 10
- UI frameworks and design systems
- DaisyUI components and themes
- Instructor code-along
- Guided exploration
- Coding exercise and peer review
- Exit ticket

CONNECTING

WE ARE ESSENTIAL TO ALBERTA

NAIT

From Tailwind utilities to design systems

- Tailwind gives flexibility, but can lead to repeated patterns
- UI frameworks provide consistent defaults and faster assembly

Today: we'll layer DaisyUI on top of Tailwind, and talk about when that trade-off is worth it.

QUICK POLL

WE ARE ESSENTIAL TO ALBERTA | NAIT

What are the trade-offs between custom CSS and prebuilt frameworks?

A. Speed vs control
B. Consistency vs uniqueness
C. Learning curve vs long-term maintainability
D. All of the above

WHAT IS DAISYUI?

A Tailwind-based UI component library

- Built on top of Tailwind CSS
- Adds semantic component classes (`btn`, `card`, `navbar`, etc.)
- Includes themes out of the box

DaisyUI doesn't replace Tailwind. It builds on it.

WHY USE A UI FRAMEWORK?

Common benefits

- Faster prototyping
- Consistent look and feel
- Fewer styling decisions per component
- Solid accessibility baselines (when used correctly)

TRADE-OFFS TO CONSIDER

What you give up

- Less granular control by default
- You inherit the framework's opinions
- Risk of “samey” UI if you don't customize

The skill is choosing when to use it.

COMPONENT CATEGORIES

What to look for in the docs

- Layout and navigation (navbar, menu)
- Containers (card, collapse)
- Inputs (input, select, checkbox)
- Feedback (alert, badge)

Pick components that replace repeated markup.

THEMES

WE ARE ESSENTIAL TO ALBERTA

NAIT

Theme-driven styling

- Themes affect colors and component appearance
- Switch theme values to quickly compare options
- Applied via a `data-theme` attribute

In our project, we set the theme from React (layout wrapper).

GUIDED EXPLORATION

Refactoring your app

Find three DaisyUI components that could improve your current app.

For each, answer:

- What problem does this solve?
- Which existing markup could it replace?
- What would you still need Tailwind utilities for?

HANDS-ON PRACTICE

Exercise: Refactor your existing exercise project to make use of the DaisyUI library, where appropriate

Objective: Implement a front-end component styling library

- Obtain the exercise starter files
  1. Update at least one existing component to use DaisyUI components
  2. Apply a theme and confirm it changes the UI
  3. Ask questions if you are unsure of what to do or how the code works.

Lesson Summary

- In this lesson, we covered the following:
  1. Install and configure DaisyUI in a React + Tailwind project
  2. Use DaisyUI prebuilt components to accelerate UI development
  3. Customize themes and integrate DaisyUI components into existing layouts

- Complete lesson 11 checklist

EXIT TICKET

Submit:

- One screenshot or snippet showing DaisyUI component integration
- One screenshot or snippet showing theme customization

Be ready to share:

- One benefit of DaisyUI
- One drawback compared to custom Tailwind styling